

GroupMemBench

GroupMemBench 笔记

论文: (<https://arxiv.org/abs/2605.14498>)

作者: Jingbo Yang、Kwei-Herng Lai、Xiaowen Wang、Shiyu Chang、Yaar Harari、
Evgeniy Gabrilovich

机构: UC Santa Barbara、Microsoft

代码: [UCSB-NLP-Chang/GroupMemBench](#)

数据: [kimperyang/GroupMemBench](#)

一、motivation

1. 现有 Agent 记忆主要为单人对话设计

LLM Agent 的长期记忆通常做三件事:

1. 从历史对话中提取值得保存的信息;
2. 把重复或冲突的信息整理成较短的记忆;
3. 面对新问题时检索相关记忆, 再交给模型回答

这套流程大多假设只有一个用户。此时“用户是谁”几乎不用判断, 历史消息也可以近似看成一条按时间排列的文本流

真实工作场景却经常是多人频道: 产品、工程、支持和管理人员在多个帖子与回复线程中协作。相同的一句话由不同人说出, 含义和可信度可能不同; 相同的问题由不同人提出, 也可能需要不同答案

2. 多人群聊新增的三类困难

2.1 群体结构: 不能只把消息排成一条长文本

群聊包含新帖子、楼中楼回复、跨项目交流、争论和共识形成。消息之间的 `reply_to` 关系说明它在回应什么

如果系统只按时间把消息拼接起来, 它可能知道“周一发送市场材料”和“今晚做压力测试”, 却不知道这两项工作分别属于谁, 也不知道后一句是在支持还是反驳前一句

当前记忆系统能否处理线程、多人回复和跨话题关系, 而不是把群聊压平成文本?

群聊不是一串消息, 系统要知道谁在回复谁、不同讨论怎么关联

2.2 说话人归属: 同一事实可能属于不同人的认知

假设 Alice 负责市场材料，Bob 负责负载测试，Carol 负责客服培训。三个人都问：“发布前我还需要做什么？”正确答案应当不同

同一问题由不同人提出，答案可能不同，所以系统必须记清谁说了什么、谁在提问？

2.3 受众适配语言：不同角色会用不同词描述同一件事

人会根据听众的专业背景调整说法，这种能力在论文中称为 **Theory of Mind (ToM, 心智理论)**。这里不需要把它理解成心理学模型；它实际表示“说话时会猜对方知道什么，再选择对方熟悉的词”

例如工程师可能说 具体工程问题，管理者可能只说对齐工作要求。系统不能只匹配表面词语，还要知道这些表达在当前团队语境中是否指向同一件事

不同角色会用不同词表达同一件事，系统还要能跨表达找到并理解正确证据？

二、方法

图结构保存：

1.场景设定

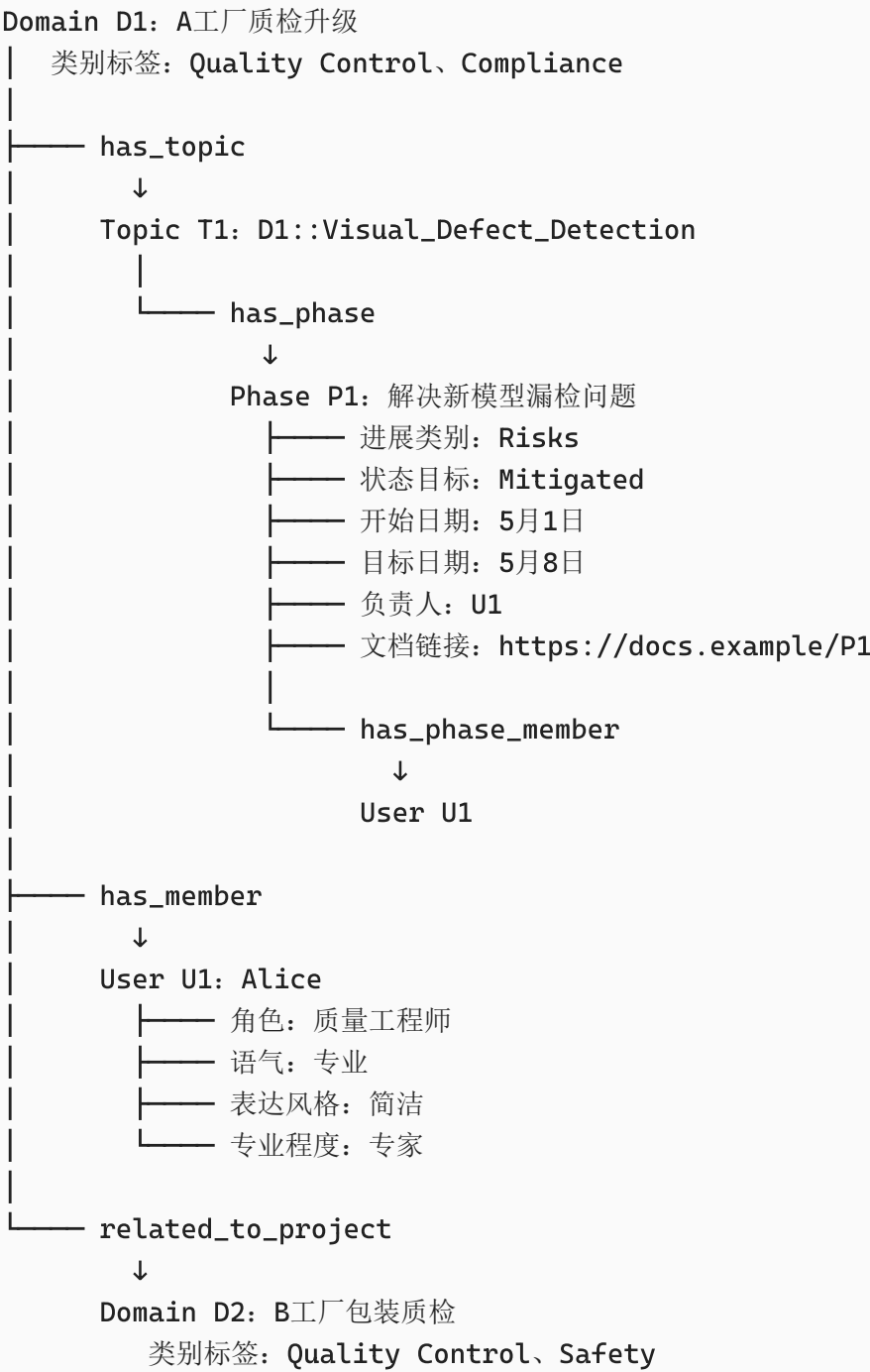
初始图包含四类静态节点：

- **Domain (项目)**：表示一个项目。附录明确说明其保存项目的类别标签 (category tags)，类别标签用于判断不同项目是否相关
- **Topic (主题)**：表示对Domain的主题划分。附录没有给出其他属性列表，只规定其节点标识采用 `d::topic_label` 格式，避免不同项目中的同名主题发生冲突
- **Phase (阶段)**：表示Topic下具有时间范围的子问题，包含进展类别、状态目标、开始日期、目标日期、阶段负责人和虚拟文档链接
- **User (用户)**：表示项目参与者，包含角色、语气、表达风格和专业程度

四类静态节点之间的关系：

- `Domain --has_topic--> Topic`：项目包含主题；
- `Topic --has_phase--> Phase`：主题被拆分为阶段；
- `Phase --belongs_to_project--> Topic`：从阶段返回所属主题的反向指针；
- `User --involved_in--> Domain`：用户参与项目；
- `Domain --has_member--> User`：项目包含用户；
- `User --active_in_phase--> Phase`：用户活跃于某个阶段；
- `Phase --has_phase_member--> User`：阶段包含参与用户；
- `Domain --related_to_project--> Domain`：共享类别标签的项目之间建立跨项目连接

初始图结构（自拟示例）



对应的反向关系:

User U1 — involved_in —> Domain D1

User U1 — active_in_phase —> Phase P1

Phase P1 — belongs_to_project —> Topic T1

生成过程中的动态扩展:

每生成一条群聊消息，系统就加入一个Message节点。Message保存生成后的消息文本、作者、时间戳和路径类型，并建立三种动态关系：

- Message --authored_by--> User : 连接消息作者;
- Message --posted_in--> Phase : 连接消息所属阶段;
- Message --reply_to--> Message : 连接被回复的父消息

2.逐条生成群聊消息

2.1基于路径的交流场景采样

这一阶段的目的

每生成一条消息前,系统先在知识图中采样一条路径。路径只负责确定交流事件的结构,包括:

- 交流发生在哪个项目、主题和阶段;
- 谁是发言者;
- 是否回复某条已有消息;
- 是否涉及跨项目交流。

这一阶段只生成交流事件的骨架,尚未生成具体消息文本

三种路径的默认采样比例为: 0.20:0.60:0.20

① 场景一: 发布新帖

路径:

```
Domain → Topic → Phase → User
```

选择过程:

1. 优先选择尚无消息的Domain, 否则均匀选择;
2. 从Domain中均匀选择一个Topic;
3. 根据“新帖需求权重”选择Phase;
4. 从Phase的活跃成员中均匀选择作者;
5. 根据Phase已有消息数量确定新帖用途。

Phase选择偏好:

- 完全没有消息的Phase优先级最高;
- 有消息但没有新帖的Phase次之;
- 新帖较少、时间上处于中期的Phase更容易被选择。

新帖用途:

```
空Phase:  
kickoff / milestone
```

消息较少：

question / update / coordination

消息较多：

update / question / blocker / decision
/ milestone / escalation / coordination

输出骨架示例：

A工厂质检升级

→ 视觉缺陷检测

→ 解决模型漏检问题

→ Alice

→ blocker

②场景二：线程回复

路径：

Domain → Topic → Phase → Parent Message → User

选择过程：

1. 优先选择当前消息量较少的Domain；
2. 根据阶段热度选择Phase；
3. 从Phase历史中选择一条父消息；
4. 从Phase的活跃成员中选择回复者。

Phase选择偏好：

- 已经有一定讨论量；
- 存在尚未回复的新帖；
- 时间上处于阶段中期。

父消息选择偏好：

- 已经形成一定讨论的消息更容易继续收到回复；
- 较新的消息更容易被回复；
- 原始新帖的权重是普通回复的2倍；

- 设置饱和机制，避免所有回复集中到同一条消息。

回复者选择规则：

- 从Phase活跃成员中均匀选择；
- 条件允许时排除父消息作者；
- 条件允许时排除最近三名发言者。

输出骨架示例：

```
A工厂质检升级
→ 视觉缺陷检测
→ 解决模型漏检问题
→ 父消息M1
→ Bob
```

③场景三：跨项目交流

路径：

```
源用户
→ 源Phase
→ 源Topic
→ 源Domain
→ 相关目标Domain
→ 目标Topic
→ 目标Phase
→ 目标用户
```

选择过程：

1. 均匀选择一个源用户；
2. 沿用户参与关系找到源Phase、Topic和Domain；
3. 沿一条 `related_to_project` 边进入相关项目；
4. 优先选择讨论较活跃的目标Phase；
5. 优先选择与源用户角色相同的目标用户；
6. 检查源Phase与目标Phase的时间顺序。

时间约束：

源Phase较早或同时发生
→ 目标Phase较晚或同时发生

如果目标Phase严格早于源Phase，则拒绝这条路径，避免未来经验反向影响过去

输出骨架示例：

Alice（A工厂质量工程师）
→ A工厂模型漏检Phase
→ related_to_project
→ B工厂包装质检Phase
→ David（B工厂质量工程师）

输出：
路径类型：post reply cross_project_reply

路径	输出节点
新帖	Domain、Topic、Phase、作者User
回复	Domain、Topic、Phase、Parent Message、回复者User
跨项目	源User、源Phase、源Domain、目标Domain、目标Phase、目标User

2.2 组装当前消息的情境上下文

场景图构建完成后，系统不会一次性生成整段群聊，而是每轮只生成一条消息，再把它写回图中
根据论文附录A，**Message** 节点自身明确存储4项信息：

字段	含义
rendered_text	GPT-5最终生成的消息正文
author	消息作者
timestamp	消息时间戳
path_type	生成路径类型

path_type 有三种：
post 新帖
reply 普通回复
cross_project_reply 跨项目回复

Message —authored_by—→ User
Message —posted_in—→ Phase
Message —reply_to—→ Parent Message

2.1完成路径采样后，系统已经确定消息将发布到哪个Phase、由谁发送，以及回复场景中的父消息。接下来，系统围绕**目标Phase**收集历史和阶段进展，组成当前消息的情境上下文 (S_u)
目标Phase是消息最终发布的位置：

新帖：路径选中的Phase
回复：父消息所在且新回复将发布的Phase
跨项目回复：相关项目中的目标Phase

① 收集目标Phase的消息历史

系统读取目标Phase最近的消息，原文窗口上限为50条：

Recent verbatim window $W = 50$

如果Phase不足50条，则使用现有消息，不需要补齐。例如只有4条消息时，就将这4条全部作为近期历史

如果还存在更早的历史，系统可以从更早窗口中抽取3个短片段。论文没有说明旧片段的具体选择算法和触发条件

② 加入Phase Memory

为了防止长对话不断重复相似内容，系统为每个Phase维护两类内部辅助记忆：

Phase Memory
├ Active Sub-issue
└ Ledger

Active Sub-issue

一个Phase第一次被使用时，系统调用GPT-5生成6个不同且互不重叠的讨论子问题。例如，“解决模型漏检问题”可以拆成：

1. 哪些缺陷类型出现漏检？
2. 是否缺少困难训练样本？
3. 是否需要调整检测阈值？
4. 新模型能否按期上线？
5. 谁负责重新训练和验收？
6. 是否需要回退旧模型？

生成后续消息时，当前子问题会作为讨论重点。附录D给出的子问题引导率为 (0.70)，但论文没有详细说明剩余情况如何选择讨论内容

Ledger

每新增8条消息，系统调用GPT-5更新一次阶段账本。更新主要参考最近16条消息，摘要不超过180词，包含：

Decided

已经确定的事项

Open Questions

尚未解决的问题及重复出现次数

Pending Commitments

已经承诺但尚未完成的工作

如果某个问题已经得到回答，更新时需要把它从 Open Questions 移动到 Decided
如果Phase尚不足8条消息，常规Ledger更新还没有触发；论文没有说明第一次更新前是否存在初始Ledger

③ 可选地加入兄弟Phase信息

系统以 (0.25) 的概率，从同一项目的另一个Phase近期历史中抽取一句信息，加入当前上下文
例如：

当前Phase：模型漏检排查

兄弟Phase：上线验收

加入的信息：

“验收团队要求周五前提交新的测试报告。”

这使不同Phase之间能够产生信息依赖

④ 对上下文去重

系统把消息历史、当前子问题、Ledger和可选的兄弟Phase信息拼接后，使用序列匹配方法进行去重：

Deduplication threshold = 0.50

目的是删除高度相似的片段，减少后续消息对已有内容的机械复述

⑤ 得到情境上下文

最终的情境上下文可以概括为：

- S_u
- └ 目标Phase最近最多50条消息
 - └ 可选的3个较早历史片段
 - └ 当前Active Sub-issue
 - └ Phase Ledger
 - └ 25%概率加入的兄弟Phase信息

2.2的作用是同时保留局部聊天历史、阶段整体进展和少量跨阶段信息，使下一条合成消息既能承接当前讨论，又不至于反复改写同一件事

2.3 基于情境上下文生成消息

它负责把：

- 2.1 选出的交流路径；
 - 2.2 组装好的情境上下文；
- 真正转化成一条群聊消息，然后把消息保存回知识图中

输入	内容
(S_u)	2.2组装的情境上下文
(C_u)	发言者画像：角色、语气、风格、专业程度
stage	Phase当前处于 early、middle 或 late
path	post、reply 或 cross-project
Phase信息	Topic、Phase名称、状态目标、目标日期
Parent Message	仅回复场景需要

第一步：程序选择Prompt模板

程序根据2.1得到的路径类型选择模板：

- ```
post
 → Channel-post Prompt

reply
 → Threaded-reply Prompt

cross_project_reply
 → Cross-project Prompt
```

例如路径是普通回复，程序就选：

**Background:** 作者和Phase背景  
**Phase:** 阶段、状态目标和日期  
**Channel history:** 情境上下文 S\_u  
**Parent message:** 要回复的父消息  
**Task:** 回应父消息并推动讨论  
**Voice:** 作者的语气、风格和专业程度  
**Constraints:** 长度和措辞要求

第二步：加入真实性控制

程序还会随机决定是否加入一些特殊要求：

| 机制                 | 触发范围       | 概率      | 作用              |
|--------------------|------------|---------|-----------------|
| Noise              | 消息         | 约5%     | 加入细微错误、旧信息或跑题内容 |
| Disagreement       | 仅回复消息      | 25%     | 对前面的观点提出实质性异议   |
| Style imperfection | 消息         | 约20%    | 加入错字、残句、自我修正等   |
| Ultra-short        | 仅回复消息      | 30%     | 强制生成1—6个词的短回复   |
| Tone jitter        | 消息         | 10%—20% | 稍微改变用户原本的语气     |
| Decision change    | 符合条件的Phase | 约5%     | 中途推翻之前作出的决定     |

这些随机开关由**程序选择**，GPT-5只负责根据选中的要求生成对应文本

第三步：调用GPT-5生成消息

论文用以下公式表示：

把情境上下文、作者画像、Phase进度和交流场景填入Prompt，然后让GPT-5生成一条候选消息  $\hat{m}$

例如：

当前Phase：模型漏检排查  
当前Sub-issue：补充夜间场景数据  
Ledger：  
- Decided：增加夜间测试集  
- Open：由谁整理漏检案例？  
- Pending：Alice周五提交报告

父消息：  
“夜间样本的误报率还没统计。”

作者：

Bob，测试工程师，直接、简洁、专家

生成结果：

“我今天补跑夜间样本，周四前把误报率和漏检案例一起发出来。”

## 第四步：把消息写回图中

生成后创建一个新的 Message 节点：

```
Message M25
├ rendered_text:
│ “我今天补跑夜间样本.....”
├ author: Bob
├ timestamp: 2026-05-06T10:20:00
└ path_type: reply
```

并建立动态边：

```
M25 —authored_by→ Bob
M25 —posted_in→ 模型漏检排查 Phase
M25 —reply_to→ 父消息 M18
```

其中：

- authored\_by：谁说的；
- posted\_in：在哪个Phase中说的；
- reply\_to：回复哪条消息；新帖没有这条边

## 2.4合成群聊质量评估

这一阶段不是继续生成消息，而是检查前面生成的整批对话是否足够像真实工作群聊。

### 评估对象

作者比较三类对话：

- ① 图结构引导的合成群聊
  - └ 本文方法生成的四个领域数据
- ② Single-Prompt Baseline
  - └ 不使用知识图，直接用一个大Prompt生成群聊

使用什么评估？

作者采用经过调整的 **G-Eval**，让GPT-5充当评审模型

每个维度评分范围是：

$s_k \in [0, 5]$

最终平均分为：

$S_{\text{avg}} = \frac{1}{6} \sum_{k=1}^6 s_k$

六个评价维度

| 维度                   | 实际检查什么             |
|----------------------|--------------------|
| Naturalness          | 消息读起来是否像真人工作群聊     |
| Coherence            | 前后消息是否连接自然、逻辑一致    |
| Diversity            | 内容、表达风格和参与者是否足够多样  |
| Contextual Relevance | 消息是否与项目、Phase及前文相关 |
| Momentum             | 对话是否推动问题解决，而不是原地重复 |
| Engagingness         | 不同参与者是否真正互动、回应和追问  |

3.构造对抗性评测问题

① Anchor Selection：选择锚点消息

程序先用轻量规则查找包含目标实体或事实的候选消息

例如：

“Carol： 供应商A的交付时间改成下周一。”

然后由LLM判断：

这条消息是否包含一个明确、可以提问并回答的事实？

不明确的消息会被排除

② Initial Composition：生成初始问题

一个针对特定问题类型的LLM，根据：

- 锚点消息；

- 问题类型
- 标准答案

LLM自动选择证据、确定候选答案

| 问题类型                    | 测试能力             |
|-------------------------|------------------|
| Multi-Hop Reasoning     | 串联多条消息才能得到答案     |
| Knowledge Update        | 识别最新信息，避免回答旧信息   |
| Term Ambiguity          | 根据提问者身份理解歧义词     |
| User-Implicit Reasoning | 理解“我”“我的”等第一人称指代 |
| Temporal Reasoning      | 根据时间戳和时间表达进行推理   |
| Abstention              | 对群聊中没有的信息拒绝回答    |

生成初始问题

锚点：交付时间改成下周一

目标答案：下周一

问题：供应商A现在什么时候交付？

③ Solve–Judge–Refine：解答—判断—改写循环

这是最重要的一步：

初始问题

↓

基线Agent检索群聊并回答

↓

LLM Judge检查回答

如果基线Agent答对：

说明问题太简单

→ LLM继续改写，让问题更困难

如果基线Agent答错：

说明问题已经足够困难

→ 接受为Benchmark题目

这里的设计目标是：**保留能够难住一个有能力的检索基线的问题**

④ **Output：保存最终题目**

每道接受的问题保存：

|                |           |
|----------------|-----------|
| question       | 最终问题      |
| gold_answer    | 标准答案      |
| asking_user_id | 谁在提问      |
| evidence_trace | 每轮使用的证据记录 |

三、实验

主实验

Table 1. Performance of various methods on the Multi-Hop Question Answering dataset.

| Method                 | Multi-Hop    | Update       | Ambiguity    | Implicit     | Temporal     | Abstention   | Average      |
|------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| RAG-based Methods      |              |              |              |              |              |              |              |
| BM25                   | <u>40.11</u> | <u>25.23</u> | 14.15        | 40.82        | <b>54.94</b> | <u>77.98</u> | <u>43.22</u> |
| text-embedding-3-large | 36.26        | 23.36        | 21.70        | <b>46.94</b> | <u>32.72</u> | 75.23        | 38.04        |
| GraphRAG [32]          | 12.09        | 14.02        | 19.81        | 14.29        | 5.56         | 66.97        | 20.56        |
| Agent Memory Systems   |              |              |              |              |              |              |              |
| Mem0 [9]               | 21.98        | 4.67         | 11.32        | 20.41        | 16.67        | <b>82.57</b> | 25.73        |
| MemGPT [28]            | 22.53        | 17.76        | 20.75        | 28.57        | 12.42        | <u>77.98</u> | 28.15        |
| A-Mem [29]             | 35.16        | 22.43        | 26.42        | <b>46.94</b> | 23.46        | 67.89        | 35.10        |
| HippoRAG [19]          | 39.56        | <b>27.10</b> | <u>30.19</u> | <u>42.86</u> | 29.63        | 75.23        | 39.72        |
| Hindsight [8]          | <b>42.31</b> | 17.76        | <b>37.74</b> | 40.82        | <b>54.94</b> | 77.06        | <b>46.01</b> |

| 实验环节            | 使用模型                          | 作用                   |
|-----------------|-------------------------------|----------------------|
| 记忆写入（Ingestion） | <b>GPT-4o-mini</b>            | 抽取事实、总结消息、构建记忆节点或知识图 |
| 向量化与稠密检索        | <b>text-embedding-3-large</b> | 将消息或记忆转成向量，计算语义相似度   |
| 检索后的问答          | <b>GPT-5</b>                  | 根据各基线检索出的记忆回答问题      |
| 准确性裁判           | <b>GPT-5</b>                  | 比较预测答案和标准答案，输出正确或错误  |

| 模块           | 输入                 | 输出                     |
|--------------|--------------------|------------------------|
| GPT-5问答Agent | 提问用户、问题、Top-10检索消息 | 推理文本+最终答案              |
| GPT-5 Judge  | 问题、标准答案、Agent最终答案  | 判分理由+Correct/Incorrect |

把原始群聊消息交给记忆系统，让系统把它们加工成以后可以检索和使用的长期记忆  
原始群聊消息

- BM25：保存原文，建立关键词索引
- Embedding：把原文转换成向量
- Mem0：抽取成“交付时间=下周一”
- A-MEM：写成记忆笔记并建立关联
- GraphRAG：提取实体和关系
- Hindsight：将消息整合成经验记忆

| 问题类型                    | 测试能力             |
|-------------------------|------------------|
| Multi-Hop Reasoning     | 串联多条消息才能得到答案     |
| Knowledge Update        | 识别最新信息，避免回答旧信息   |
| Term Ambiguity          | 根据提问者身份理解歧义词     |
| User-Implicit Reasoning | 理解“我”“我的”等第一人称指代 |
| Temporal Reasoning      | 根据时间戳和时间表达进行推理   |
| Abstention              | 对群聊中没有的信息拒绝回答    |

性能—效率权衡气泡图

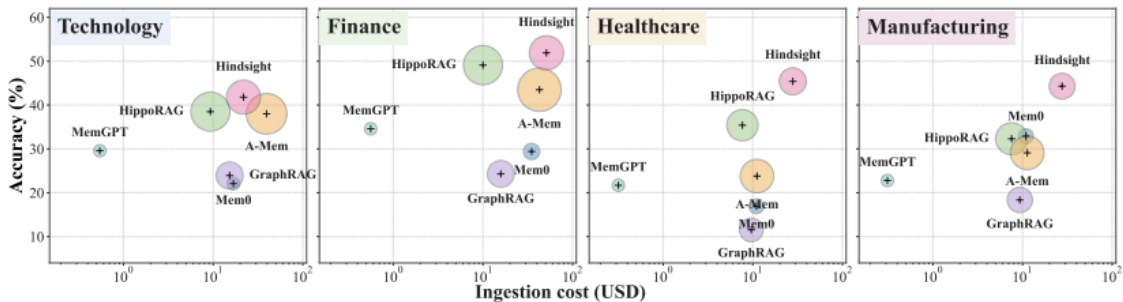
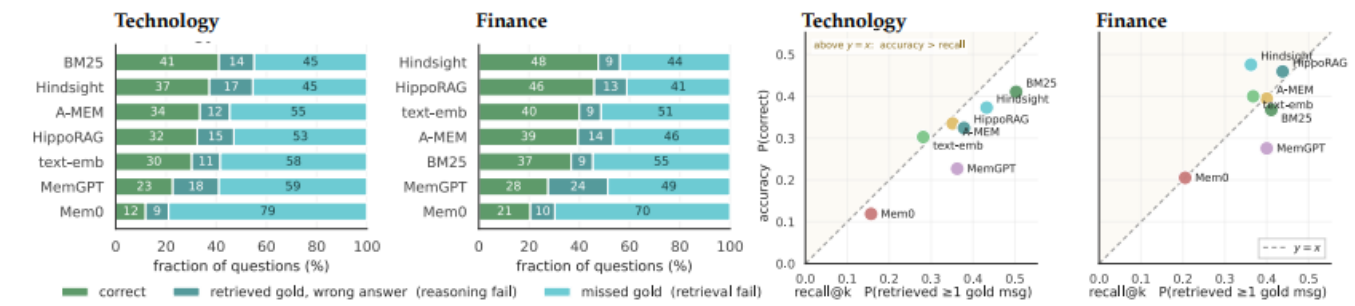


Figure 4: Performance–efficiency trade-off across the four domains. Each marker is one of six baselines (five agent memory systems plus GRAPHRAG):  $x$ -axis is ingestion cost (USD, log scale),  $y$ -axis is per-domain QA accuracy (%), and bubble area encodes on-disk storage in GB. Per-(baseline, domain) numerical values are listed in Table 11.



| 图中元素 | 含义                                              |
|------|-------------------------------------------------|
| 四个子图 | Technology、Finance、Healthcare、Manufacturing四个领域 |
| 横轴   | 建立记忆的写入成本，单位为美元                                 |
| 纵轴   | 该领域的问答准确率                                       |
| 气泡大小 | 记忆库占用的磁盘空间                                      |
| 一个气泡 | 一种记忆系统                                          |

失败模式分解与检索召回率—回答准确率分析



- #问题
- 1.问题精炼会不会把题从"难"改成"病态", 不符合正常难度, 以致最终分数偏低?
  - 2.拒答是因为确实没有检索到还是模型检索到了但是能力不够
  - 强模型会不会即使检索不到也编造, 弱模型能力弱检索不到反而高
  - 3.未来信息泄漏问题
- BM25基线测评时, 讲对话全部输入之后再进行的提问, 如果遇到时间推理类问题, 会不会影响准确性